

# 구현 소스 코드 - 주요 코드 작성

## RAG 기반 법률 상담 챗봇 Back-end 구현 코드 요약

| 구분    | 내용                                                           |
|-------|--------------------------------------------------------------|
| 작성 범위 | JWT 인증 처리, 로그인/로그아웃, WebSocket 실시간 채팅, Kakao OAuth 로그인       |
| 주요 기술 | FastAPI, SQLAlchemy, PostgreSQL, JWT, WebSocket, Kakao OAuth |
| 작성 기준 | Legal-Chat-Bot/Back-end main branch 핵심 소스 코드 발췌 및 설명         |

### 구현 범위 요약

- security.py: 비밀번호 해싱, JWT Access/Refresh Token 생성, 현재 사용자 검증, WebSocket 토큰 검증 처리
- login.py: 이메일 로그인, 중복 로그인 방지, 토큰 발급, 로그아웃 시 토큰 블랙리스트 및 세션 제거 처리
- websocket/chat.py: JWT 기반 WebSocket 연결 검증, 채팅방 생성/조회, 메시지 저장, AI 답변 저장 및 전송 처리
- oauth/kakao.py: 카카오 인가 코드 기반 로그인, 사용자 정보 조회, 신규 회원 자동 가입, 서비스 JWT 발급 처리

※ 제출용 문서에 맞춰 전체 파일이 아닌 핵심 로직만 발췌하였다.

# 1. 보안 및 JWT 토큰 처리

인증 공통 로직은 core/security.py 에 분리하였다. REST API 와 WebSocket 에서 공통으로 사용하는 비밀번호 검증, JWT 생성, 토큰 디코딩, 사용자 조회를 담당한다.

| 항목           | 구현 내용                                                   |
|--------------|---------------------------------------------------------|
| 비밀번호 처리      | passlib CryptContext 로 hash/verify 처리                   |
| 토큰 생성        | payload 에 exp, type, jti 를 포함하여 Access/Refresh Token 발급 |
| 인증 검증        | HTTPBearer Depends 와 token blacklist 를 통해 로그아웃 토큰 차단    |
| WebSocket 검증 | WebSocket 연결 시 token query 를 decode 하고 사용자 조회           |

## 주요 코드 1-1. JWT 생성 및 비밀번호 검증 | app/core/security.py

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
security = CustomHTTPBearer()

def verify_password(plain_password: str, hashed_password: str):
    return pwd_context.verify(plain_password, hashed_password)

def create_token(data: dict, expires_delta: timedelta, token_type: str):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + expires_delta
    to_encode.update({
        "exp": expire,
        "type": token_type,
        "jti": str(uuid.uuid4()),
    })
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_access_token(data: dict) -> str:
    return create_token(data, timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES), "access")

def create_refresh_token(data: dict) -> str:
    return create_token(data, timedelta(minutes=REFRESH_TOKEN_EXPIRE_MINUTES), "refresh")
```

## 주요 코드 1-2. 현재 사용자 / WebSocket 토큰 검증 | app/core/security.py

```
def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security),
    db: Session = Depends(get_db)):
    token = credentials.credentials
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    email = payload.get("sub")
    jti = payload.get("jti")

    if token_store.is_blacklisted(jti):
        raise HTTPException(status_code=401, detail="로그아웃된 토큰입니다.")

    user = db.query(User).filter(User.email == email).first()
    if user is None:
        raise HTTPException(status_code=401, detail="사용자를 찾을 수 없습니다.")
    return user

async def verify_ws_token(token: str, db: Session) -> User | None:
    payload = decode_token(token)
    email = payload.get("sub")
    jti = payload.get("jti")
    if not email or not jti or token_store.is_blacklisted(jti):
        return None
    return db.query(User).filter(User.email == email).first()
```

## 2. 로그인 / 로그아웃 처리

로그인 로직은 사용자 조회, 비밀번호 검증, 중복 로그인 확인, Access/Refresh Token 발급 순서로 진행된다. 로그아웃 시에는 jti 를 blacklist 에 등록하고 active session 을 제거한다.

| 기능    | 구현 내용                                                          |
|-------|----------------------------------------------------------------|
| 로그인   | 이메일 조회 → 비밀번호 검증 → 중복 로그인 확인 → JWT 발급                          |
| 로그 기록 | AUTH_001, AUTH_002, AUTH_003 등 실패 사유를 logger 에 저장              |
| 로그아웃  | 토큰 decode → 사용자 조회 → 카카오 로그아웃 처리 → blacklist/active session 제거 |

### 주요 코드 2-1. 로그인 처리 | app/routes/auth/login.py

```
@router.post("/login", response_model=Token, summary="로그인")
def login(login_data: LoginRequest, request: Request, db: Session = Depends(get_db)):
    trace_id = get_trace_id(request)
    user = db.query(User).filter(User.email == login_data.email).first()

    if not user:
        create_log(db, level=Level.WARN, status_code=401,
            error_code="AUTH_001", message="이메일 없음",
            trace_id=trace_id)
        raise HTTPException(status_code=401, detail="이메일이 올바르지 않습니다.")

    if not verify_password(login_data.password, user.password):
        create_log(db, level=Level.WARN, status_code=401,
            error_code="AUTH_002", message="비밀번호 불일치",
            user_id=user.user_id, trace_id=trace_id)
        raise HTTPException(status_code=401, detail="비밀번호가 올바르지 않습니다.")

    user_id = str(user.user_id)
    if token_store.has_active_session(user_id):
        raise HTTPException(status_code=409, detail="이미 로그인된 계정입니다.")

    token_data = {"sub": user.email, "user_id": user_id}
    access_token = create_access_token(data=token_data)
    refresh_token = create_refresh_token(data=token_data)
    token_store.set_active_session(user_id, decode_token(access_token)["jti"])
    return Token(access_token=access_token, refresh_token=refresh_token, token_type="bearer")
```

### 주요 코드 2-2. 로그아웃 및 토큰 무효화 | app/routes/auth/login.py

```
@router.post("/logout", summary="로그아웃")
async def logout(request: Request,
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: Session = Depends(get_db)):
    token = credentials.credentials
    payload = decode_token(token)
    jti = payload.get("jti")
    exp = payload.get("exp")
    user_id = payload.get("user_id")

    user = db.query(User).filter(User.user_id == user_id).first()
    if user.social == SocialType.KAKAO:
        await kakao_logout(user.social_id)

    token_store.blacklist_token(jti, exp)
    token_store.remove_active_session(user_id)
    return {"message": "로그아웃 되었습니다."}
```

### 3. WebSocket 실시간 채팅 처리

실시간 채팅은 /ws/chat 에서 처리한다. 연결 시 JWT 를 검증하고, session\_id 가 있으면 기존 채팅방 조회, 없으면 첫 메시지를 기준으로 새 채팅방을 생성한다.

| 단계     | 처리 내용                                                |
|--------|------------------------------------------------------|
| 연결 검증  | token 쿼리 파라미터를 verify_ws_token 으로 검증하고 실패 시 4001 종료  |
| 세션 처리  | session_id 가 있으면 사용자 소유 채팅방 조회, 없으면 첫 메시지로 새 Chat 생성 |
| 메시지 저장 | 질문을 Message 에 먼저 저장하고 process_chat 결과를 answer 로 갱신   |
| 예외 처리  | 토큰 만료, 블랙리스트, 빈 메시지, 없는 세션, 서버 오류를 구분하여 응답           |

주요 코드 3-1. 연결 및 채팅방 검증 | app/routes/websocket/chat.py

```
@router.websocket("/ws/chat")
async def websocket_chat(websocket: WebSocket,
                          token: str = Query(...),
                          session_id: Optional[str] = Query(None),
                          db: Session = Depends(get_db)):
    trace_id = uuid.uuid4().hex
    user = await verify_ws_token(token, db)

    if not user:
        create_log(db, level=Level.WARN, endpoint="/ws/chat", method="WS",
                  status_code=401, error_code="WS_AUTH_001",
                  message="토큰 검증 실패", trace_id=trace_id)
        await websocket.close(code=4001)
        return

    await websocket.accept()
    chat_session = None

    if session_id:
        chat_session = db.query(Chat).filter(
            Chat.session_id == session_id,
            Chat.user_id == user.user_id,
        ).first()
    if not chat_session:
        await websocket.send_json({"type": "error", "code": "SESSION_NOT_FOUND"})
        await websocket.close(code=4004)
        return
```

### 주요 코드 3-2. 메시지 수신 / 세션 생성 / 답변 전송 | app/routes/websocket/chat.py

```
data = await websocket.receive_json()
message_text = (data.get("message") or "").strip()

if not message_text:
    await websocket.send_json({"type": "error", "code": "EMPTY_MESSAGE"})
    continue

if chat_session is None:
    chat_session = Chat(user_id=user.user_id, title=message_text[:20])
    db.add(chat_session)
    db.commit()
    db.refresh(chat_session)

msg = Message(session_id=chat_session.session_id,
               user_id=user.user_id,
               question=message_text,
               answer="")
db.add(msg)
db.commit()

a_response = await process_chat(db=db,
                                user_id=str(user.user_id),
                                session_id=str(chat_session.session_id),
                                question=message_text)

msg.answer = a_response.answer
db.commit()

await websocket.send_json({
    "type": "message",
    "session_id": str(chat_session.session_id),
    "message_id": str(msg.message_id),
    "answer": msg.answer,
})
```

## 4. Kakao OAuth 로그인 처리

카카오 OAuth 는 프론트엔드가 받은 인가 코드를 백엔드로 전달하고, 백엔드가 카카오 토큰과 사용자 정보를 조회한 뒤 서비스 자체 JWT 를 발급하는 구조이다.

| 단계         | 처리 내용                                         |
|------------|-----------------------------------------------|
| 로그인 URL 발급 | /auth/kakao/login-url 에서 카카오 인증 페이지 URL 반환    |
| 인가 코드 처리   | 프론트에서 전달한 code 로 카카오 access token 발급          |
| 사용자 조회/가입  | 카카오 ID 기준 기존 회원 조회, 없으면 자동 회원가입               |
| 서비스 토큰 발급  | 카카오 토큰이 아닌 서비스 Access/Refresh Token 을 발급하여 반환 |

### 주요 코드 4-1. 카카오 로그인 URL / Callback 처리 | app/routes/oauth/kakao.py

```
@router.get("/login-url")
def kakao_login_url(force_login: bool = False):
    login_url = get_kakao_login_url(force_login=force_login)
    return {"login_url": login_url}

@router.post("/callback", response_model=KakaoLoginResponse)
async def kakao_callback(body: KakaoCallbackRequest, db: Session = Depends(get_db)):
    kakao_token = await get_kakao_access_token(body.code)
    kakao_user = await get_kakao_user_info(kakao_token["access_token"])

    kakao_id = kakao_user["kakao_id"]
    email = kakao_user.get("email")
    nickname = kakao_user.get("nickname", "카카오유저")

    user = db.query(User).filter(
        User.social == SocialType.KAKAO,
        User.social_id == kakao_id,
    ).first()
```

### 주요 코드 4-2. 신규 회원 자동 가입 및 서비스 JWT 발급 | app/routes/oauth/kakao.py

```
is_new_user = False
if not user:
    if email:
        existing_user = db.query(User).filter(User.email == email).first()
        if existing_user:
            raise HTTPException(status_code=409,
                                detail="이미 해당 이메일로 가입된 계정이 있습니다.")

    user = User(email=email or f"kakao_{kakao_id}@kakao.local",
                password="",
                name=nickname,
                social=SocialType.KAKAO,
                social_id=kakao_id)
    db.add(user)
    db.commit()
    db.refresh(user)
    is_new_user = True

access_token = create_access_token(data={"sub": user.email,
                                         "user_id": str(user.user_id)})
refresh_token = create_refresh_token(data={"sub": user.email,
                                           "user_id": str(user.user_id)})

return KakaoLoginResponse(access_token=access_token,
                           refresh_token=refresh_token,
                           is_new_user=is_new_user)
```

# 5. 구현 흐름 정리

주요 구현 코드가 실제 서비스 흐름에서 어떤 역할을 담당하는지 정리한 내용이다.

| 서비스 흐름    | 연결 코드                                   | 역할                                                 |
|-----------|-----------------------------------------|----------------------------------------------------|
| 사용자 로그인   | routes/auth/login.py + core/security.py | 사용자 검증 후 Access/Refresh Token 발급                   |
| 인증 API 보호 | core/security.py:get_current_user       | HTTP Authorization Header 의 JWT 검증 및 사용자 조회        |
| 실시간 채팅    | routes/websocket/chat.py                | WebSocket 연결 인증, 세션 생성/조회, 메시지 저장/응답 전송            |
| 소셜 로그인    | routes/oauth/kakao.py                   | 카카오 인가 코드 처리, 사용자 정보 조회, 신규 회원 자동 가입               |
| 로그아웃      | routes/auth/login.py:logout             | 토큰 blacklist 등록, active session 제거, 카카오 유저 로그아웃 처리 |

## 구현상 특징

- 인증 로직을 core/security.py 로 분리하여 REST API 와 WebSocket 에서 공통 재사용할 수 있도록 구성하였다.
- Access Token 과 Refresh Token 을 분리하고, jti 를 사용해 개별 토큰을 식별할 수 있도록 하였다.
- token\_store 를 통해 중복 로그인 방지, 로그아웃 토큰 블랙리스트, 활성 세션 제거를 처리하였다.
- WebSocket 채팅은 session\_id 유무에 따라 기존 채팅방 입장 또는 새 채팅방 생성을 분기하였다.
- Kakao OAuth 는 카카오 access token 을 프론트로 직접 넘기지 않고, 백엔드에서 사용자 정보를 조회한 뒤 서비스 JWT 를 발급하는 방식으로 구성하였다.